Week – 2

CASE STUDY: BANK ACCOUNT AND MONEY TRANSFERS

Problem Statement

A retail banking organization wants to develop a Java-based banking application that allows customers to maintain accounts, deposit money, withdraw money, and transfer funds between accounts.

Recently, the bank has experienced several operational problems:

- Customers are withdrawing amounts beyond their permitted overdraft limits.
- Money is deducted from one account but is not credited to the destination account when a system failure occurs during a transfer.
- Frozen accounts are still accepting withdrawal requests.
- Account balances do not always match the corresponding transaction history.
- Some transactions are processed with zero or negative amounts.

The development team has proposed a basic implementation using public fields, primitive data types, and separate service methods.

Your team is required to analyse these problems and propose an improved object-oriented domain model for the banking application.

Questions

Q1. Identification of Domain Invariants (6 Marks)

Identify at least six business invariants that must be maintained in the banking application.

For each invariant:

- State the business rule clearly.
- Explain why the rule is important.
- Identify the possible consequences if the rule is violated.

No.	Business rule (invariant)	Why it is important	Consequence if violated
1	Transaction amounts must be greater than zero.	Prevents invalid transactions.	Zero or negative transactions may cause incorrect balances.
2	Account balance must not go below the permitted overdraft limit.	Protects the bank and customers from excessive withdrawals.	Customers may withdraw more money than allowed.
3	Frozen accounts must not allow withdrawals or transfers.	Protects accounts from unauthorized transactions.	Money may be withdrawn from a blocked account.
4	Closed accounts must not accept deposits, withdrawals, or transfers.	Ensures closed accounts cannot be used for new transactions.	Incorrect transactions may occur in closed accounts.
5	A transfer must debit the source account and credit the destination account for the same amount.	Ensures money is transferred correctly.	Money may be lost or created incorrectly.
6	Account balance must match the transaction ledger.	Maintains accurate financial records.	The balance and transaction history may show different amounts.
7	Account numbers must uniquely identify accounts.	Helps identify the correct account.	Transactions may affect the wrong account.
8	Every completed transaction must have a unique transaction ID and a permanent record.	Helps track and verify transactions.	Duplicate transactions and missing records may occur.

These invariants should be enforced inside the domain classes instead of relying only on external service methods.

Q2. Entities and Value Objects (6 Marks)

Identify the important domain concepts involved in the banking application.

Classify the following concepts as entities or value objects:

- Bank Account
- Account Number
- Money
- Ledger Entry
- Overdraft Limit
- Fund Transfer

Justify your classification based on identity, equality, lifecycle and mutability.

An entity is an object that has a unique identity and continues to exist even when its attributes change. A value object is identified by its values rather than by a unique identity.

Domain concept	Classification	Justification
Bank Account	Entity	Has a unique account number, balance and lifecycle. Its balance changes over time, but its identity remains the same.
Account Number	Value Object	Represents the identity value of an account. It should be immutable and compared by its value.
Money	Value Object	Contains an amount and currency. Two Money objects with the same amount and currency are considered equal.
Ledger Entry	Entity	Has a unique transaction ID and represents a specific transaction record that must remain traceable.
Overdraft Limit	Value Object	Represents the permitted overdraft amount. It is compared by value and should not be changed arbitrarily.
Fund Transfer	Entity	Has a unique transfer ID and a lifecycle, such as initiated, completed or failed.

Conclusion: Entities are identified by their unique IDs, whereas value objects are compared by their values. Value objects should generally be immutable to prevent accidental changes.

Q3. Self-Enforcing Objects and Java Design (8 Marks)

Consider the following operations:

- Deposit money.
- Withdraw money.
- Freeze an account.
- Unfreeze an account.
- Close an account.

Design suitable Java classes and methods to implement these operations.

Explain how your design ensures that:

- Invalid transaction amounts are rejected.
- Accounts cannot be withdrawn from when they are frozen or closed.
- The overdraft limit is respected.
- Account data cannot be modified directly by external classes.

Self-enforcing objects are objects that protect their own data and prevent invalid operations. In Java, this can be achieved using encapsulation, private fields, validation and methods.

```java
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

enum AccountStatus {
    ACTIVE, FROZEN, CLOSED
}

class Money {
    private final long amountInPaise;
    private final String currency;

    public Money(long amountInPaise, String currency) {
        if (amountInPaise < 0) {
            throw new IllegalArgumentException("Invalid amount");
        }
        if (currency == null || currency.isBlank()) {
            throw new IllegalArgumentException("Invalid currency");
        }
        this.amountInPaise = amountInPaise;
        this.currency = currency;
    }

    public long getAmountInPaise() {
        return amountInPaise;
    }

    public String getCurrency() {
        return currency;
    }

    public boolean isPositive() {
        return amountInPaise > 0;
    }
}
```

```
class LedgerEntry {
    private final String transactionId;
    private final String transactionType;
    private final Money amount;
    private final String timestamp;

    public LedgerEntry(String type, Money amount) {
        this.transactionId = UUID.randomUUID().toString();
        this.transactionType = type;
        this.amount = amount;
        this.timestamp = java.time.Instant.now().toString();
    }

    public String getTransactionId() {
        return transactionId;
    }

    public String getTransactionType() {
        return transactionType;
    }

    public Money getAmount() {
        return amount;
    }

    public String getTimestamp() {
        return timestamp;
    }
}

class BankAccount {
    private final String accountNumber;
    private long balance;
    private final long overdraftLimit;
    private AccountStatus status;
```

```java
class BankAccount {

    public BankAccount(String number, long initialBalance,
                       long overdraftLimit) {
        if (number == null || number.isBlank()
                || initialBalance < 0 || overdraftLimit < 0) {
            throw new IllegalArgumentException("Invalid account details");
        }

        this.accountNumber = number;
        this.balance = initialBalance;
        this.overdraftLimit = overdraftLimit;
        this.status = AccountStatus.ACTIVE;
    }

    public void deposit(Money money) {
        checkActive();
        if (!money.isPositive()
                || !money.getCurrency().equals("INR")) {
            throw new IllegalArgumentException("Invalid deposit");
        }

        balance = Math.addExact(balance, money.getAmountInPaise());
        ledger.add(new LedgerEntry(type: "DEPOSIT", money));
    }

    public void withdraw(Money money) {
        checkActive();
        if (!money.isPositive()
                || !money.getCurrency().equals("INR")) {
            throw new IllegalArgumentException("Invalid withdrawal");
        }

        long amount = money.getAmountInPaise();
        if (amount > balance + overdraftLimit) {
            throw new IllegalArgumentException("Overdraft limit exceeded");
        }
```

```
    balance = Math.subtractExact(balance, amount);
    ledger.add(new LedgerEntry(type: "WITHDRAWAL", money));
}

public void freeze() {
    if (status == AccountStatus.CLOSED) {
        throw new IllegalStateException("Account is closed");
    }
    status = AccountStatus.FROZEN;
}

public void unfreeze() {
    if (status != AccountStatus.FROZEN) {
        throw new IllegalStateException("Account is not frozen");
    }
    status = AccountStatus.ACTIVE;
}

public void close() {
    if (status != AccountStatus.ACTIVE || balance < 0) {
        throw new IllegalStateException("Cannot close account");
    }
    status = AccountStatus.CLOSED;
}

private void checkActive() {
    if (status != AccountStatus.ACTIVE) {
        throw new IllegalStateException(
            "Account is frozen or closed");
    }
}
```

```
    public long getBalance() {
        return balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public AccountStatus getStatus() {
        return status;
    }

    public List<LedgerEntry> getLedger() {
        return List.copyOf(ledger);
    }
}

public class BankingApp {
    Run | Debug
    public static void main(String[] args) {
        BankAccount account =
            new BankAccount(number: "1001", initialBalance: 1000000, overdraftLimit: 50000);

        account.deposit(new Money(amountInPaise: 20000, currency: "INR"));
        account.withdraw(new Money(amountInPaise: 10000, currency: "INR"));

        System.out.println("Balance: Rs. "
            + account.getBalance() / 100.0);
        System.out.println("Status: " + account.getStatus());
    }
}
```

1. Invalid amounts: The `Money` class rejects negative values, and deposit and withdrawal methods reject zero amounts.
2. Frozen or closed accounts: The `checkActive()` method prevents deposits and withdrawals when an account is not active.
3. Overdraft limit: The withdrawal method checks whether the withdrawal exceeds the available balance plus the permitted overdraft.
4. Encapsulation: All account fields are private. External classes must use methods to perform operations.
5. Account status: The `AccountStatus` enum represents active, frozen and closed states.
6. Ledger: Every successful deposit or withdrawal creates a transaction record.

Q4. Money Transfer and Aggregate Consistency (6 Marks)

Consider a situation where a customer transfers ₹5,000 from Account A to Account B.

During the transfer, the system successfully deducts the amount from Account A but crashes before crediting Account B.

Answer the following:

1. What problems can arise?

If ₹5,000 is deducted from Account A but not credited to Account B, the customer loses access to ₹5,000. The account balances become incorrect, and the transaction history may not match the actual balances.

2. Which business invariants are violated?

- The source and destination accounts must be updated consistently.
- The total transferred amount must be preserved.
- Account balances must match the transaction ledger.
- Every completed transfer must have a corresponding transaction record.

3. How would you design the transfer operation?

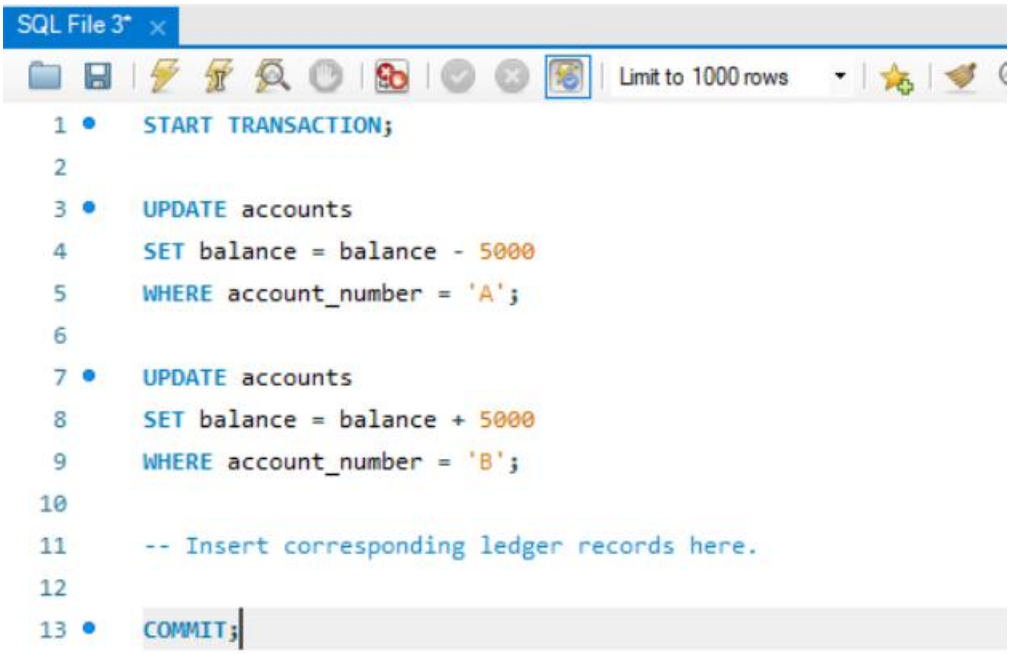
The transfer should be performed as one atomic database transaction. Both the debit and credit must succeed together, or neither should take effect.

The process is:

1. Validate the amount and both account statuses.
2. Check the source account's available balance and overdraft limit.
3. Deduct ₹5,000 from Account A.
4. Credit ₹5,000 to Account B.
5. Record the transaction in the ledger.
6. Commit the transaction only after all operations succeed.

4. How would your design handle failures?

If any operation fails, the database transaction should be rolled back. This restores the original balances and prevents partial transfers.



If an error occurs, execute `ROLLBACK` instead of `COMMIT`. In a real application, validation and balance checks must also be performed safely within the transaction, and the application must verify that both updates affected the intended accounts.

5. What if the accounts belong to different banking systems?

A single database transaction usually cannot control both banks. The systems may use a distributed transaction protocol or a Saga pattern.

In a Saga pattern, each bank performs its part of the transfer, and if a later step fails, a compensating transaction is initiated to reverse the earlier operation. Unique transfer IDs, status tracking, retries and reconciliation help prevent duplicate transfers and unresolved transactions.

Q5. Ledger Management and Design Evaluation (4 Marks)

1. What information should a ledger entry contain?

A ledger entry should contain the following information:

- Transaction ID: Unique identification of the transaction.
- Transaction type: Deposit, withdrawal or transfer.
- Amount: The amount involved in the transaction.
- Timestamp: Date and time of the transaction.
- Account number: The account associated with the transaction.

- Transaction status: Whether the transaction is pending, successful or failed.
- Balance after transaction: The account balance after the transaction, if maintained by the system.

2. Should an existing ledger entry be modified or deleted?

No. A completed ledger entry should not be modified or deleted because it provides a permanent record of financial activities.

If a mistake occurs, the bank should create a new correcting or reversing entry. This maintains transparency, accountability and a reliable audit trail.

3. How would you ensure that the account balance remains consistent with the transaction history?

The bank should update the account balance and ledger entries within the same database transaction. Both operations should succeed together or be rolled back.

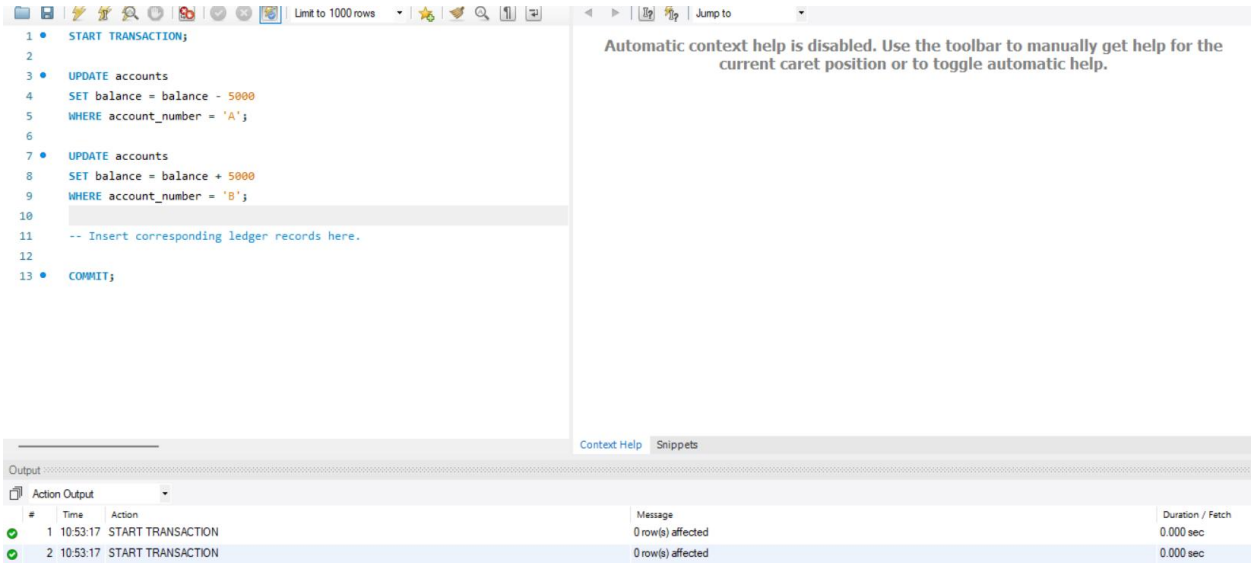
The system should also use unique transaction IDs, database constraints, regular reconciliation and audit checks to detect inconsistencies.

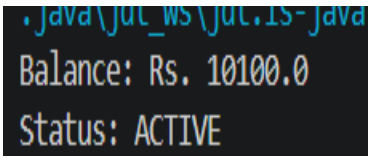
4. What challenges might arise when the transaction history becomes very large?

- Storage: A large number of transactions requires more database space.
- Performance: Searching and retrieving transaction records may become slower.
- Reporting: Generating statements and reports may take more time.
- Maintenance: Backups, auditing and data recovery become more complex.

These challenges can be managed using database indexing, pagination, archiving older records, partitioning and optimized queries.

Output -





Conclusion

The banking application can be made safer by using object-oriented programming concepts such as encapsulation, entities, value objects and self-enforcing objects. Business invariants prevent invalid transactions, while atomic database transactions and immutable ledger entries maintain accurate balances and reliable transaction histories.